Thato Phenyo Molete

BLCH9X2-Blockchain

Assignment 3: Transaction Engine Lab

Girhub repository: https://github.com/phenyotmolete/BLCH9X2-Block_Chain/tree/assignment3

1. Overview:

Aim: To implement the transaction (payment-message) layer of the simple blockchain. The transaction layer sits between the block construction lab and the wallet and digital signature lab. A transaction is a structured message that proposes a transfer of value from a sender to a recipient, recorded at a point in time, and authenticated by a digital signature.

In this assignment a minimal five-field transaction schema is defined, and subsequently implemented, created, serialised and deserialise usingJSON. Lastly a mempool is built with structural validation. A demonstration of acceptance and rejection will be made.

2. Design:

2.1 Framework

The transaction class executes the five required fields:

Field	Type	Role
sender	str	Payer identity / address
recipient	str	Payee identity / address
amount	float	Value transferred, must be > 0
timestamp	float	Numeral value representing time of creation
signature	str | None	Authorisation placeholder

2.2 Selection: Account model vs UTXO

For this assignment, I opted for an account-style framework (sender / recipient / amount) instead of a full UTXO framework. This selection is a sort of emulation of the Ethereum-style ledgers, while also being the simpler applicable design format. The mapping to Bitcoin's native UTXO framework is:

I. sender mapped onto authority spending prior unspent outputs
II. recipient mapped onto new output locking script (address)
III. amount mapped onto output value
IV. timestamp mapped onto not consensus-critical in Bitcoin's UTXO model
V. signature mapped on to unlocking script (witness)

The account-style model purposefully excludes explicit inputs referencing prior outputs, multiple outputs (or change addresses within a single transaction), and script-based spending predicates beyond /pay to address/, which are mostly included in the real UTXO.

2.3 The numeric type in which the amount is stored:

I used a float value to denote the amounts. However, I do recognise that floats can create small rounding errors when dealing with decimal values. In a real remittance or stablecoin system, I would opt to rather use either integer minor units or Python's Decimal type to ensure that monetary amounts are handled precisely. Using a float to store transaction amounts presents a limitation because floating-point numbers cannot always exactly represent decimal values. For example, a calculation such as 0.4 + 0.3 may result in 0.300000000000001 instead of giving us the exact value of 0.3. Although these differences

are generally negligibly small, they can become problematic in a financial system, especially where we have transaction amounts which need to match exactly. This can be problematic in particular when reconciling transactions or performing multiple calculations and currency conversions. Therefore, although using float is acceptable for this context, a production remittance or stablecoin system would typically use integer minor units, such as cents, or Python's Decimal type to avoid rounding errors.

2.4 Serialisation:

Serialisation is a necessary process to perform because serialisation converts the Python object/data that we have into a format that can be stored, transmitted, or processed consistently.

In this case, we have a transaction stored as a Python object, such as a dictionary. Serialisation converts the transaction into a standard format like JSON. After serialisation this transaction data can then be transmitted or stored. So for example, if my Python dictionary has a transaction stored as {"sender": "Phenyo", "receiver": "ProfMba", "amount": 310}. Serialisation converts this Python object into a standard format such as JSON: {"sender":"Phenyo","receiver":"ProfMba","amount":310}. In the standard format, the transaction can be easily transmitted or stored. This ensures that different computers use the same representation of the same transaction**.**

Serialisation is especially important when we're dealing with transaction ledgers because the transaction needs to be represented exactly the same way each time. If two computers represent the same transaction differently, they could produce different hashes later, even though the underlying transaction is identical.

Function:	Example:
Json.dumps() Takes the Python transaction and turns it into JSON text.	Python data: {"receiver": "ProfMba", "amount": 600, "sender": "Phenyo"} json.dumps() converts this to: {"receiver":"ProfMba","amount":600,"sender":"Phenyo"}
sort_keys=True Ensures the keys are always put in the same order.	The transaction will always be in the order: {"receiver":"ProfMba","amount":600,"sender":"Phenyo"}
Separators=(“,”,”:”) Removes unnecessary spaces so that the data representation is consistent.	Rather than: {"amount": 600, "receiver": "ProfMba"} we get: {"amount":100,"receiver":"ProfMba"}

Serialisation generates a canonical, deterministic byte representation. However, serialisation is not in itself is not a demonstration of authentication. Serialisation does not prove who created the transaction because a JSON dump is packaging, rather than proof of authorship. But it is still required so that two nodes agree on identical bytes for the same logical transaction.

2.5 Structural validation:

Structural validation is used to check that a transaction contains all the required information and that each piece of information is in the correct basic format.

The system checks whether the transaction is properly performed before accepting it. So if we think about it like a form at a bank, before a payment is accepted, the system checks:

Input data:	Requirement:	Example:
Sender or recipient	Cannot be blank, missing or whitespace only. If it is blank, whitespace or missing, the transaction is rejected from the mempool.	“Phenyo” is accepted but “ ” or “” is not.
Amount	Must be a number and must be greater than zero, otherwise the transaction is rejected from the mempool.	20 is accepted, but "twenty", 0, or -60 is not. Bool amounts are also rejected because in Python, True and False are treated as integers (True == 1 and False == 0), even though they obviously aren't valid transaction amounts.
Timestamp	The transaction must have a timestamp that can be interpreted as a number, allowing the system to know when the transaction occurred. Otherwise the transaction rejected from the mempool.	A number like 1723970000 is accepted.

Structural validation does not purposefully check signature authenticity, sender balance / overdraft, replay protection, or compliance rules (sanctions, corridor limits), these require wallets, a chain with account state, and consensus.

2.6 Mempool and mutation safety:

Mempool.get_all() returns a shallow copy of the internal transaction list, so callers cannot mutate the mempool's internal state by editing the returned list. This is exercised in the test code by changing an unrelated transaction (Evan -> Hayley) to the returned snapshot and confirming the mempool's own length is unaffected.

3. Implementation:

Operation:	Function:
Creation of transaction:	Transaction(sender, recipient, amount, timestamp=None, signature=None)
Conversion between transaction object and python dictionary:	Transaction.to_dict() /Transaction.from_dict(data)

Conversion between transaction in python dictionary form into JSON representation:	Transaction.to_json() /Transaction.from_json(payload)
Structural validation:	validate_transaction(tx) -> bool
Mempool functions:	Mempool.add(tx)- the system validates the transaction first and if it is valid, it is added to the mempool and the output we see is “true” Mempool.get_all() -> list – here we can observe all of the transactions currently waiting in the mempool len(mempool)- tells us how many transactions are currently in the mempool Mempool.clear()- removes all transactions from the mempool.

4. Execution evidence:

4.1 Test output:

```
test_schema_has_five_fields: PASS
test_json_round_trip_preserves_fields: PASS
test_dict_round_trip: PASS
test_valid_transaction_accepted: PASS
test_negative_amount_rejected: PASS
test_zero_amount_rejected: PASS
test_empty_sender_rejected: PASS
test_empty_recipient_rejected: PASS
test_whitespace_only_recipient_rejected: PASS
test_non_numeric_amount_rejected: PASS
test_bool_amount_rejected: PASS
test_mempool_accepts_valid_and_rejects_invalid: PASS
test_mempool_get_all_returns_copy: PASS

All 13 tests passed.
```

4.2 Demo script output:

```
1) Phenyo -> Thato, amount = 10
JSON payload: {"amount":10.0,"recipient":"Thato","sender":"Phenyo","signature":null,"timestamp":1787035881.463265}
Accepted: True  |  Mempool size: 1

2) Molete -> Thato, amount = -5 (invalid: negative amount)
Accepted: False  |  Mempool size: 1

3) Phenyo -> '' (invalid: empty recipient)
Accepted: False  |  Mempool size: 1

4) Round-trip check: serialise Phenyo's tx, then rebuild it
Restored: Transaction('Phenyo' -> 'Thato', amount=10.0, signature=None)
Round-trip preserved all fields correctly.

Final mempool contents (should contain only Phenyo -> Thato)
  {'sender': 'Phenyo', 'recipient': 'Thato', 'amount': 10.0, 'timestamp': 1787035881.463265, 'signature': None}

Demo complete: only the valid Phenyo -> Thato transaction is in the mempool.
```

5. Reflection:

Collision and pre-image resistance (from Assignment 1) protect the integrity of the linked chain of blocks that will eventually contain this transaction. However, hashing alone provides no confidentiality and, at this stage, no proof of who authorised the transfer. Validation, as implemented here, is necessary but not sufficient for payment safety.

Appendix: AI Declaration:

Tool: Claude (Anthropic). Usage: assisted with Python syntax for JSON serialisation patterns and docstring phrasing, following the structure and schema introduced in the Lecture 3 slides and live-coding notebook (03_Live_Coding.ipynb) from the lecture material.